WEEK 3

DAA ASSIGNMENT

Kruskal's Algorithm

NAME – SUJAL GAHLAWAT

SAPID- 590021805

Kruskal's Algorithm is a greedy technique that builds the Minimum Spanning Tree (MST) of a weighted, connected, undirected graph by repeatedly picking the cheapest available edge.

It begins by sorting every edge in the graph from lowest to highest weight. Edges are then examined one at a time in that order, and an edge is added to the growing tree only if including it would not close a cycle among the vertices already connected.

This selection process keeps going until the tree being built holds exactly V − 1 edges, at which point every vertex is connected and the MST is complete.

Basic steps

1. Arrange all the edges of the graph in ascending order of weight.
2. Pick the edge that currently has the lowest weight.
3. Verify whether including this edge would form a cycle in the tree built so far.
4. If no cycle is formed, accept the edge into the spanning tree; otherwise discard it.
5. Continue this selection process until the tree contains V − 1 edges in total.

Spanning Tree

A spanning tree of a connected, undirected graph is a sub-graph that touches every vertex of the original graph while containing no cycles at all.

Whenever a graph has V vertices, any spanning tree drawn from it will always consist of precisely V − 1 edges.

Important properties

- Every vertex of the original graph is included.
- No cycles are present anywhere in the tree.
- Exactly one unique path exists between any pair of vertices.
- A single connected graph may yield more than one valid spanning tree.

C CODE OF KRUSKAL'S ALGORITHM

```
#include <stdio.h>
#define MAX 100

struct Edge {
    int u, v, weight;
};
```

```
int parent[MAX];

int find(int i)
{
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int u, int v)
{
    int rootU = find(u);
    int rootV = find(v);
    parent[rootU] = rootV;
}

void sortEdges(struct Edge edges[], int n)
{
    struct Edge temp;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (edges[j].weight > edges[j + 1].weight)
            {
                temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

int main()
{
    int V, E;
    struct Edge edges[MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("Enter number of edges: ");
    scanf("%d", &E);
    printf("Enter edges (source destination weight):\n");
    for (int i = 0; i < E; i++)
    {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].weight);
    }

    for (int i = 0; i < V; i++)
        parent[i] = i;

    sortEdges(edges, E);

    int count = 0;
    int totalWeight = 0;

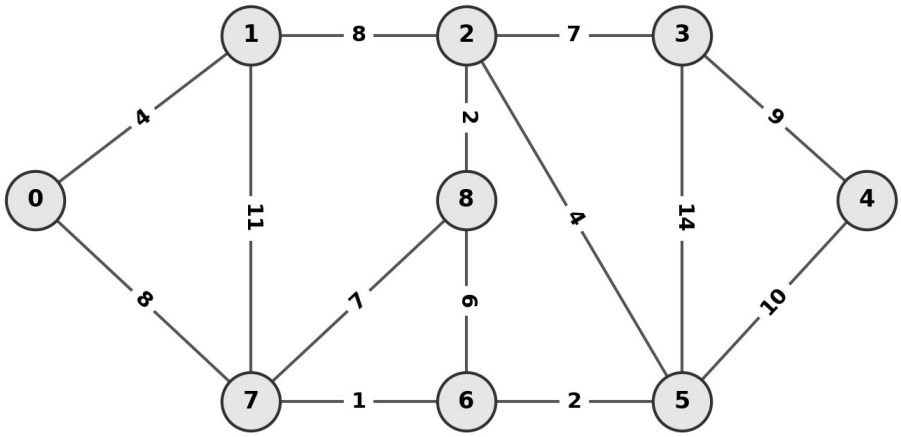
    printf("\nEdges in Minimum Spanning Tree:\n");
    for (int i = 0; i < E && count < V - 1; i++)
```

```
    {
        int u = edges[i].u;
        int v = edges[i].v;

        if (find(u) != find(v))
        {
            printf("%d -- %d = %d\n", u, v, edges[i].weight);
            totalWeight += edges[i].weight;
            unionSet(u, v);
            count++;
        }
    }

    printf("Total weight of MST = %d\n", totalWeight);
    return 0;
}
```

Graph used



Sample run and output

```
Enter number of vertices: 5
Enter number of edges: 6
Enter edges (source destination weight):
0 1 1
1 3 2
0 3 3
0 2 4
2 3 5
3 4 5

Edges in Minimum Spanning Tree:
0 -- 1 = 1
1 -- 3 = 2
0 -- 2 = 4
3 -- 4 = 5
Total weight of MST = 12
```